

EXPERIMENT 3: GREEDY ALGORITHMS IMPLEMENTATION & ANALYSIS

AIM

To design, implement, and analyze greedy approach algorithms by solving three standard computational problems with interactive user input: Fractional Knapsack Problem, Activity Selection Problem, and Maximum Events Attended (Greedy Interval Scheduling with Min-Heap).

THEORETICAL BACKGROUND

A greedy algorithm builds up a solution piece by piece, always choosing the next piece that offers the most immediate and local benefit (locally optimal choice) with the intent of reaching a global optimum. For a greedy strategy to guarantee a globally optimal result, the problem must exhibit two core properties:

- **1. Greedy-Choice Property:** A globally optimal solution can be arrived at by making locally optimal (greedy) choices without reconsidering previously made decisions.
- **2. Optimal Substructure:** An optimal solution to the overall problem contains within it optimal solutions to its subproblems.

Problem Breakdown:

- **Part 1** — Fractional Knapsack: Items have weights and values. Fractions of items are permissible. The greedy choice selects items in descending order of value-to-weight ratio (v/w). This guarantees the maximum value within knapsack capacity W .
- **Part 2** — Activity Selection: A set of n activities with start and finish times. To maximize the count of mutually non-overlapping activities, sort activities by earliest finish time. Selecting the activity that finishes earliest leaves maximum remaining time for subsequent activities.
- **Part 3** — Maximum Number of Events Attended: Given events with $[start, end]$ days, only one event can be attended per day. Sort events by start day, and for each day, greedily attend the active event having the earliest closing deadline (using a Min-Heap/Priority Queue).

ALGORITHMIC DESIGN & PSEUDOCODE

Part 1: Fractional Knapsack Algorithm

- Calculate ratio $r_i = \text{value}_i / \text{weight}_i$ for all items $i = 1$ to n .
- Sort all items in descending order of ratio r_i .
- Initialize $\text{total_value} = 0$, $\text{remaining_capacity} = W$.
- Iterate through sorted items: if item weight $\leq$ remaining_capacity, take 100% of item. Else, take fraction ($\text{remaining_capacity} / \text{weight}$), add fraction * value to total_value, and terminate.

Part 2: Activity Selection Algorithm

- Sort activities in ascending order of their finish times f_i .
- Select the first activity (index 0) and set $\text{last_finish_time} = f_0$.
- Iterate through remaining activities: if start_time $s_i \geq \text{last_finish_time}$, select activity i and update $\text{last_finish_time} = f_i$.

Part 3: Maximum Events Attended (Min-Heap Strategy)

- Sort events by their start day.
- Track current day d, starting from the minimum start day to maximum end day.
- Push end days of all events starting on day d into a min-priority queue.
- Discard events from the priority queue whose end day < d (expired).
- If the queue is non-empty, pop the event with earliest end day, attend it, and increment count.

IMPLEMENTATION CODE

Part 1: Fractional Knapsack Algorithm

```
import java.util.*;

public class AssignCookies {
    public static int findContentChildren(int[] greed, int[] cookies) {
        Arrays.sort(greed);
        Arrays.sort(cookies);

        int child = 0;
        int cookie = 0;

        while (child < greed.length && cookie < cookies.length) {
            if (cookies[cookie] >= greed[child]) child++;
            cookie++;
        }

        return child;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of children: ");
        int n = sc.nextInt();
        int[] greed = new int[n];

        System.out.println("Enter greed factor of each child:");
        for (int i = 0; i < n; i++) {
            greed[i] = sc.nextInt();
        }

        System.out.print("Enter number of cookies: ");
        int m = sc.nextInt();
        int[] cookies = new int[m];

        System.out.println("Enter cookie sizes:");
        for (int i = 0; i < m; i++) {
            cookies[i] = sc.nextInt();
        }
    }
}
```

```

        int result = findContentChildren(greed, cookies);
        System.out.println("Maximum number of content children: " + result);

        sc.close();
    }
}

```

Part 2: Activity Selection Algorithm

```

import java.util.*;

public class NonOverlappingIntervals {
    public static int eraseOverlapIntervals(int[][] intervals) {
        if (intervals.length == 0) {
            return 0;
        }

        // Sort by ending time
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));

        int selected = 1;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.length; i++) {
            // Non-overlapping interval
            if (intervals[i][0] >= lastEnd) {
                selected++;
                lastEnd = intervals[i][1];
            }
        }

        // Number of intervals that must be removed
        return intervals.length - selected;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of intervals: ");
        int n = sc.nextInt();
        int[][] intervals = new int[n][2];

        System.out.println("Enter start and end time of each interval:");
        for (int i = 0; i < n; i++) {
            System.out.print("Interval " + (i + 1) + ": ");
            intervals[i][0] = sc.nextInt();
            intervals[i][1] = sc.nextInt();
        }

        int result = eraseOverlapIntervals(intervals);
    }
}

```

```

        System.out.println("Minimum number of intervals to remove: " + result);

        sc.close();
    }
}

```

Part 3: Maximum Events Attended (Min-Heap Strategy)

```

import java.util.*;

public class MaximumNumberOfEvents {

    public static int maxEvents(int[][] events) {

        // Sort events by starting day
        Arrays.sort(events, (a, b) -> Integer.compare(a[0], b[0]));

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        int n = events.length;
        int i = 0;
        int day = 0;
        int attended = 0;

        while (i < n || !minHeap.isEmpty()) {

            // If no active events, jump to the next event's start day
            if (minHeap.isEmpty()) {
                day = Math.max(day, events[i][0]);
            }

            // Add all events that have started
            while (i < n && events[i][0] <= day) {
                minHeap.add(events[i][1]);
                i++;
            }

            // Remove events that have already expired
            while (!minHeap.isEmpty() && minHeap.peek() < day) {
                minHeap.poll();
            }

            // Attend the event that ends earliest
            if (!minHeap.isEmpty()) {
                minHeap.poll();
                attended++;
                day++;
            }
        }
    }
}

```

```
        return attended;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of events: ");
        int n = sc.nextInt();

        int[][] events = new int[n][2];

        System.out.println("Enter start and end day of each event:");

        for (int i = 0; i < n; i++) {
            System.out.print("Event " + (i + 1) + ": ");
            events[i][0] = sc.nextInt();
            events[i][1] = sc.nextInt();
        }

        int result = maxEvents(events);

        System.out.println("Maximum number of events that can be attended: " + result);

        sc.close();
    }
}
```

SAMPLE TEST CASES & RESULTS

Part 1: Fractional Knapsack Test

```
$ java AssignCookies.java
Enter number of children: 3
Enter greed factor of each child:
1 2 3
Enter number of cookies: 2
Enter cookie sizes:
1 1
Maximum number of content children: 1
```

Part 2: Activity Selection Test

```
$ java NonOverlappingIntervals.java
Enter number of intervals: 4
Enter start and end time of each interval:
Interval 1: 1 2
Interval 2: 2 3
Interval 3: 3 4
Interval 4: 1 3
Minimum number of intervals to remove: 1
```

Part 3: Maximum Events Attended Test

```
$ java MaximumNumberOfEvents.java
Enter number of events: 4
Enter start and end day of each event:
Event 1: 1 2
Event 2: 2 3
Event 3: 3 4
Event 4: 1 2
Maximum number of events that can be attended: 4
```